

Kotlin Android

FocusBloom

코드 중심 발표

집중 루틴 관리 앱의 핵심 기능과 Kotlin 구현 설명

모바일 앱 프로그래밍 최종과제



할 일
타이머
통계
추천

핵심 기능은 하나의 흐름으로 연결된다

1. 입력

할 일 제목과 집중 시간을 입력한다.
addTask()가 입력값을 검사한다.

2. 실행

선택한 할 일을 기준으로
CountDownTimer를 실행한다.

3. 저장

완료 여부와 누적 기록을
SharedPreferences에 저장한다.

4. 추천

기록을 바탕으로 다음
집중 시간을 추천한다
.

발표 핵심

각 기능을 단순히 소개하는 것이 아니라, 어떤 함수가 그 기능을 담당하는지 코드 스니펫과 함께 설명한다.

앱 시작 시 데이터와 화면을 준비하는 onCreate()

MainActivity.kt - onCreate()

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    prefs = getSharedPreferences(PREFS, MODE_PRIVATE)  
    loadTasks()  
    if (tasks.isEmpty()) seedTasks()  
    buildUi()  
    refresh()  
}
```

발표 설명 포인트

- SharedPreferences를 먼저 준비해 저장 데이터를 사용할 수 있게 한다.
- loadTasks()로 이전에 저장한 할 일 목록을 불러온다.
- 처음 실행이면 seedTasks()로 예시 데이터를 만든다.
- buildUi()와 refresh()를 나눠 화면 생성과 데이터 표시를 분리했다.

XML 없이 Kotlin 코드로 화면을 구성하는 buildUi()

MainActivity.kt - buildUi()

```
private fun buildUi() {
    val scroll = ScrollView(this)
    val root = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        setPadding(dp(18), dp(20), dp(18), dp(24))
    }
    scroll.addView(root)

    timerText = text("25:00", 48, "#2F766D", true)
    progressBar = ProgressBar(this, null,
        android.R.attr.progressBarStyleHorizontal)
    root.addView(card())
    setContentView(scroll)
}
```

발표 설명 포인트

- ScrollView 안에 LinearLayout을 두어 세로 스크롤 화면을 만든다.
- TextView, Button, EditText, ProgressBar를 Kotlin 코드에서 직접 생성한다.
- card(), text(), primaryButton() 같은 보조 함수로 반복 UI를 줄였다.
- 타이머/입력/목록/통계 영역을 하나의 화면에 배치했다.

할 일 등록은 데이터 객체와 저장 호출이 핵심이다

MainActivity.kt - FocusTask / addTask()

```
private data class FocusTask(  
    val title: String,  
    val minutes: Int,  
    var done: Boolean = false  
)  
  
private fun addTask() {  
    val title = titleInput.text.toString().trim()  
    val minutes = parseMinutes(minutesInput.text.toString().trim())  
    if (title.isEmpty()) {  
        Toast.makeText(this, "할 일을 입력하세요.",  
            Toast.LENGTH_SHORT).show()  
        return  
    }  
    tasks.add(FocusTask(title, minutes))  
    saveTasks()  
    resetTimer()  
    refresh()  
}
```

발표 설명 포인트

- FocusTask는 제목, 목표 시간, 완료 여부를 한 번에 담은 데이터 클래스다.
- 입력값이 비어 있으면 Toast로 사용자에게 알려준다.
- 목록에 추가한 뒤 saveTasks()로 즉시 저장한다.
- resetTimer()와 refresh()로 타이머와 화면을 최신 상태로 맞춘다.

CountDownTimer가 집중 세션의 중심 로직이다

MainActivity.kt - startTimer()

```
private fun startTimer() {
    if (running) return
    running = true
    timer = object : CountDownTimer(remainingMs, 1000) {
        override fun onTick(millisUntilFinished: Long) {
            remainingMs = millisUntilFinished
            updateTimerView()
        }

        override fun onFinish() {
            running = false
            remainingMs = 0
            completeSession()
            refresh()
        }
    }.also { it.start() }
}
```

발표 설명 포인트

- CountDownTimer는 1초마다 onTick()을 호출한다.
- onTick()에서 남은 시간을 저장하고 화면을 갱신한다.
- onFinish()에서 세션 완료 처리와 통계 갱신을 실행한다.
- running 값으로 중복 시작을 막아 타이머 상태를 안정적으로 관리한다.

할 일 목록을 저장하는 SharedPreferences 코드

MainActivity.kt - saveTasks() / loadTasks()

```
private fun saveTasks() {
    val array = JSONArray()
    tasks.forEach { task ->
        array.put(JSONObject()
            .put("title", task.title)
            .put("minutes", task.minutes)
            .put("done", task.done))
    }
    prefs.edit()
        .putString(KEY_TASKS, array.toString())
        .apply()
}

private fun loadTasks() {
    val array = JSONArray(prefs.getString(KEY_TASKS, "["))
    // JSON 배열을 FocusTask 목록으로 복원
}
```

발표 설명 포인트

- SharedPreferences는 간단한 로컬 저장에 적합하다.
- 목록 데이터는 JSON 배열 문자열로 변환해 저장한다.
- 앱을 다시 켜면 loadTasks()가 JSON을 읽어 객체 목록으로 복원한다.
- 별도 서버나 DB 없이도 과제 규모에 맞는 영구 저장을 구현했다.

추천 시간은 사용 기록을 반영하는 규칙 기반 로직이다

MainActivity.kt - recommendMinutes()

```
private fun recommendMinutes(): Int {
    val open = countOpenTasks()
    val sessions = prefs.getInt(KEY_COMPLETED_SESSIONS, 0)
    val total = prefs.getInt(KEY_TOTAL_MINUTES, 0)
    val average = averageMinutes(total, sessions)

    return when {
        open >= 5 -> 15
        average >= 35 -> 30
        sessions >= 3 -> 25
        else -> 20
    }
}
```

발표 설명 포인트

- 남은 할 일이 많으면 부담을 줄이기 위해 짧은 15분을 추천한다.
- 평균 집중 시간이 길면 더 긴 30분 세션을 추천한다.
- 완료 세션 수가 쌓이면 기본 추천을 25분으로 올린다.
- 단순 타이머가 아니라 사용자 기록을 반영한다는 점이 독창적이다.

마무리: FocusBloom은 할 일 관리, 타이머, 저장, 통계, 추천을 하나의 학습 루틴 흐름으로 연결한 앱이다.